TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

FIRST ORDER PREDICATE LOGIC

Lab 2 Report

Submitted By:

Name: **Chandan Kumar Shah**

Roll No: **080BCT023**

Date: **2083-03-16**

Submitted To:

Department of Electronics
and Computer Engineering
Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1 Objectives

1. To understand First Order Predicate Logic (FOPL) and its role in knowledge representation for Artificial Intelligence.
2. To translate natural-language statements into FOPL and subsequently into Prolog programs.
3. To implement rule-based reasoning using Prolog's backward chaining (unification and backtracking).
4. To demonstrate both *forward chaining* and *backward chaining* on an inference problem.
5. To analyse classic AI problems—such as the Criminal/Iraq scenario and the Monkey-Banana problem—using predicate logic.

2 Theoretical Background

2.1 First Order Predicate Logic (FOPL)

First Order Predicate Logic (also called *predicate calculus*) extends propositional logic by introducing **predicates**, **quantifiers**, and **variables**. It allows us to make precise statements about objects and their relationships.

Key components of FOPL:

- **Predicates:** Relations over objects — e.g. $\text{Mortal}(x)$, $\text{Parent}(x, y)$.
- **Constants:** Specific named objects — e.g. *john*, *iraq*, *bananas*.
- **Variables:** Placeholders — e.g. X, Y, bound by quantifiers.
- **Functions:** Map objects to objects — e.g. $\text{father\_of}(x)$.
- **Universal Quantifier** ($\forall$): "for all" — e.g. $\forall x\,(\text{Man}(x) \Rightarrow \text{Mortal}(x))$.
- **Existential Quantifier** ($\exists$): "there exists" — e.g. $\exists x\, P(x)$.

2.2 Mapping FOPL to Prolog

Prolog is a direct computational realisation of Horn-clause logic, a decidable subset of FOPL. The correspondence is summarised below:

Table 1: FOPL-to-Prolog Mapping

FOPL Construct	**Prolog Equivalent**	**Example**
Fact: $P(a)$	`p(a).`	`mortal(socrates).`
Rule: $\forall X\,(P(X) \Rightarrow Q(X))$	`q(X) :- p(X).`	`mortal(X) :- man(X).`
Conjunction: $A \wedge B$	`A, B`	`male(X), parent(X,Y)`
$\forall$ (Universal)	Capitalised variables	`criminal(X) :- ...`
$\exists$ (Existential)	Ground facts	`has_missile(iraq).`

2.3 Inference Strategies

2.3.1 Backward Chaining

Prolog's default inference strategy. Starting from the *goal*, Prolog works backwards: it finds a rule whose head matches the goal, then attempts to prove each sub-goal in the rule body. This is implemented via depth-first search with backtracking.

2.3.2 Forward Chaining

Forward chaining starts from known *facts* and repeatedly applies rules to derive new facts until the goal is reached. Although not native to Prolog, it can be simulated using `assert/1` to add derived facts to the knowledge base iteratively.

3 Program 1 — Criminal / Weapons / Iraq Problem

3.1 Problem Statement

Write the following statements in FOPL and implement them as a Prolog program. Determine whether *George is a criminal*.

1. Every American who sells weapons to hostile nations is a criminal.
2. Every enemy of America is a hostile nation.
3. Iraq has some missiles.
4. All missiles of Iraq were sold by George.
5. George is an American.
6. Iraq is a country.
7. Iraq is the enemy of America.
8. Missiles are weapons.

3.2 FOPL Representation

---

1. $\forall X\ (\text{American}(X) \land \text{SellsWeapons}(X, Y) \land \text{Hostile}(Y) \Rightarrow \text{Criminal}(X))$
2. $\forall X\ (\text{EnemyOfAmerica}(X) \Rightarrow \text{Hostile}(X))$
3. $\exists X\ (\text{Missile}(X) \land \text{Owns}(iraq, X))$
4. $\forall X\ (\text{Missile}(X) \land \text{Owns}(iraq, X) \Rightarrow \text{Sells}(george, X, iraq))$
5. $\text{American}(george)$
6. $\text{Country}(iraq)$
7. $\text{EnemyOfAmerica}(iraq)$
8. $\forall X\ (\text{Missile}(X) \Rightarrow \text{Weapon}(X))$

3.3 Prolog Implementation

```
% Facts
enemy_of_america(iraq).
has_missile(iraq).
sells_weapons(george, iraq).
american(george).
country(iraq).

% Rules
criminal(X) :-
    american(X),
    sells_weapons(X, Y),
```

```
    hostile(Y).

hostile(X) :-
    enemy_of_america(X).

hostile(X) :-
    country(X),
    enemy_of_america(X).
```

Listing 1: Criminal/Iraq problem in Prolog

3.4 Inference Trace (Backward Chaining)

```
?- criminal(george).

  Goal: criminal(george)
    Sub-goal 1: american(george)           -> FACT  => TRUE
    Sub-goal 2: sells_weapons(george, Y)
                Y = iraq                   -> FACT  => TRUE
    Sub-goal 3: hostile(iraq)
      Sub-goal 3a: enemy_of_america(iraq)  -> FACT  => TRUE
    hostile(iraq) => TRUE
  criminal(george) => TRUE
```

3.5 Output

```
?- criminal(george).
true.

?- hostile(iraq).
true.
```

Listing 2: Program 1 output

3.6 Discussion

The program encodes eight natural-language statements as FOPL formulae and then translates them into Prolog facts and rules. The key inference is a three-step backward chain: Prolog verifies that George is American, that he sells weapons to Iraq, and that Iraq is hostile (because Iraq is an enemy of America). The conjunction of all three sub-goals proves `criminal(george)`.

4 Program 2 — Monkey-Banana Problem

4.1 Problem Statement

Represent the classic Monkey-Banana problem in FOPL and prove that the monkey can reach the bananas. A room contains a monkey, a chair, and bananas hanging from the ceiling. The monkey can move the chair under the bananas, climb onto the chair, and reach them.

4.2 FOPL Representation

Objects: monkey, chair, bananas

Facts:

$$\begin{aligned}&\text{InRoom}(monkey) \quad \text{InRoom}(chair) \quad \text{InRoom}(bananas)\\&\text{Dexterous}(monkey) \quad \text{Tall}(chair)\\&\text{CanMove}(monkey, chair, bananas) \quad \text{CanClimb}(monkey, chair)\end{aligned}$$

Rules:

$$\begin{aligned}&\forall X, Y\ (\text{Dexterous}(X) \land \text{Close}(X, Y) \Rightarrow \text{CanReach}(X, Y))\\&\forall X, Y, Z\ (\text{GetOn}(X, Y) \land \text{Under}(Y, Z) \land \text{Tall}(Y) \Rightarrow \text{Close}(X, Z))\\&\forall X, Y\ (\text{CanClimb}(X, Y) \Rightarrow \text{GetOn}(X, Y))\\&\forall X, Y, Z\ (\text{InRoom}(X) \land \text{InRoom}(Y) \land \text{InRoom}(Z)\\&\qquad\qquad \land \text{CanMove}(X, Y, Z) \Rightarrow \text{Under}(Y, Z))\end{aligned}$$

4.3 Prolog Implementation

```
% Facts
in_room(bananas).
in_room(chair).
in_room(monkey).
dexterous(monkey).
tall(chair).
can_move(monkey, chair, bananas).
can_climb(monkey, chair).

% Rules
can_reach(X, Y) :-
    dexterous(X),
    near(X, Y).

near(X, Z) :-
    get_on(X, Y),
```

```
    under(Y, Z),
    tall(Y).

get_on(X, Y) :-
    can_climb(X, Y).

under(Y, Z) :-
    in_room(X),
    in_room(Y),
    in_room(Z),
    can_move(X, Y, Z).
```

Listing 3: Monkey-Banana problem in Prolog

4.4 Inference Trace (Backward Chaining)

```
?- can_reach(monkey, bananas).

  Goal: can_reach(monkey, bananas)
    Sub-goal 1: dexterous(monkey)           -> FACT  => TRUE
    Sub-goal 2: near(monkey, bananas)
      Sub-goal 2a: get_on(monkey, chair)
        Sub-goal: can_climb(monkey, chair)  -> FACT  => TRUE
      get_on(monkey, chair) => TRUE
      Sub-goal 2b: under(chair, bananas)
        Sub-goal: in_room(monkey)           -> FACT  => TRUE
        Sub-goal: in_room(chair)            -> FACT  => TRUE
        Sub-goal: in_room(bananas)          -> FACT  => TRUE
        Sub-goal: can_move(monkey,chair,bananas) -> FACT => TRUE
      under(chair, bananas) => TRUE
      Sub-goal 2c: tall(chair)              -> FACT  => TRUE
    near(monkey, bananas) => TRUE
  can_reach(monkey, bananas) => TRUE
```

4.5 Output

```
?- can_reach(monkey, bananas).
true.
```

Listing 4: Program 2 output

4.6 Discussion

The Monkey-Banana problem demonstrates multi-level rule chaining in Prolog. The query `can_reach(monkey, bananas)` triggers a four-level deep inference chain through `near`, `get_on`, and `under`. Each level requires the satisfaction of multiple sub-goals, all of which are ultimately resolved to base facts. This elegantly shows how Prolog's automatic

backtracking explores the rule dependency graph without any explicit control flow from the programmer.

5 Assignment 1 — Steve's Course Preference (Resolution)

5.1 Problem Statement

Assume the following facts:

- Steve only likes easy courses.
- Science courses are hard.
- All courses in the basket weaving department are easy.
- BK301 is a basket weaving course.

Use resolution to answer: *What course would Steve like?*

5.2 FOPL Representation

---

1. $\forall X\ (\text{likes}(steve, X) \Rightarrow \text{easy}(X)) \equiv \forall X\ (\text{course}(X) \wedge \text{easy}(X) \Rightarrow \text{likes}(steve, X))$
2. $\forall X\ (\text{science\_course}(X) \Rightarrow \text{hard}(X))$
3. $\forall X\ (\text{basket\_weaving\_course}(X) \Rightarrow \text{easy}(X))$
4. $\text{basket\_weaving\_course}(bk301)$
5. $\text{course}(bk301)$

5.3 Resolution Proof

To prove $\text{likes}(steve, bk301)$ by resolution:

1. From (4): $\text{basket\_weaving\_course}(bk301)$ — fact.
2. Apply rule (3) with $X = bk301$: $\text{basket\_weaving\_course}(bk301) \Rightarrow \text{easy}(bk301)$. $\therefore \text{easy}(bk301)$.
3. From (5): $\text{course}(bk301)$ — fact.
4. Apply rule (1) with $X = bk301$: $\text{course}(bk301) \wedge \text{easy}(bk301) \Rightarrow \text{likes}(steve, bk301)$. $\therefore \text{likes}(steve, bk301)$. ■

5.4 Prolog Implementation

```
% Facts
course(bk301).
basket_weaving_course(bk301).

% Rules
likes(steve, X) :-
    course(X),
```

```
    easy(X).

hard(X) :-
    science_course(X).

easy(X) :-
    basket_weaving_course(X).
```

Listing 5: Steve's course preference in Prolog

5.5 Output

```
?- likes(steve, X).
X = bk301.

?- easy(bk301).
true.

?- likes(steve, bk301).
true.
```

Listing 6: Assignment 1 output

5.6 Discussion

The resolution proceeds by chaining the basket-weaving fact through the "easy" rule, then combining it with the "course" fact to satisfy Steve's preference condition. The Prolog engine automatically performs this resolution through backward chaining and unification, returning `X = bk301` as the unique solution.

6 Assignment 2 — Horses and Mammals

6.1 Problem Statement

Write the following in FOPL, implement in Prolog, and test:

1. Horses are mammals.
2. An offspring of a horse is a horse.
3. Bluebeard is Charlie's parent.
4. Offspring and parents are inverse relations.
5. Every mammal has a parent.

Goal: *Is Charlie a horse?*

6.2 FOPL Representation

---

1. $\forall X\ (\text{Horse}(X) \Rightarrow \text{Mammal}(X))$
2. $\forall X, Y\ (\text{Offspring}(X, Y) \wedge \text{Horse}(Y) \Rightarrow \text{Horse}(X))$
3. $\text{Parent}(\textit{bluebeard}, \textit{charlie})$
4. $\forall X, Y\ (\text{Offspring}(X, Y) \Leftrightarrow \text{Parent}(Y, X))$
5. $\forall X\ (\text{Mammal}(X) \Rightarrow \exists Y\ \text{Parent}(Y, X))$

Additional fact needed for inference: $\text{Horse}(\textit{bluebeard})$.

6.3 Prolog Implementation

```
% Facts
parent(bluebeard, charlie).
horse(bluebeard).

% Rules
% 1. Horses are mammals
mammal(X) :- horse(X).

% 2. Offspring of a horse is a horse
horse(X) :-
    offspring(X, Y),
    horse(Y).

% 4. Offspring and parent are inverse relations
offspring(X, Y) :- parent(Y, X).
```

Listing 7: Horses and mammals in Prolog

6.4 Inference Trace

```
?- horse(charlie).

  Goal: horse(charlie)
    Sub-goal 1: offspring(charlie, Y)
      -> parent(Y, charlie)    -> Y = bluebeard  => TRUE
    Sub-goal 2: horse(bluebeard)  -> FACT  => TRUE
  horse(charlie) => TRUE

?- mammal(charlie).
  Goal: mammal(charlie)
    Sub-goal: horse(charlie)   -> (see above) => TRUE
  mammal(charlie) => TRUE
```

6.5 Output

```
?- horse(charlie).
true.

?- mammal(charlie).
true.

?- horse(bluebeard).
true.

?- offspring(charlie, bluebeard).
true.
```

Listing 8: Assignment 2 output

6.6 Discussion

The key inference path is: since Bluebeard is Charlie's parent (fact), Charlie is the offspring of Bluebeard (inverse relation, Rule 4). Since Bluebeard is a horse (fact), Charlie—being the offspring of a horse—is also a horse (Rule 2). Since Charlie is a horse, Charlie is also a mammal (Rule 1). This demonstrates how inverse relations and transitive inference work in Prolog through rule chaining.

7 Assignment 3 — Student Evaluation (Forward & Backward Chaining)

7.1 Problem Statement

Consider the following Student Evaluation System:

Rule I: If there are chances that a student will get a good position **AND** there is no other student with marks greater than him, **Then** he will get first position.

Rule II: If a student attends all his lectures regularly **AND** studies his course books, **Then** he will cover his course contents.

Rule III: If a student covers his course contents, **Then** there are chances that he will get a good position.

Known Facts:

(i) A student attends all his lectures regularly.

(ii) A student studies his course books.

(iii) There is no one with marks higher than him.

Goal: Show that this student will get first position using *forward* and *backward* chaining.

7.2 FOPL Representation

$$\begin{aligned}
\text{Rule I:} &\quad \forall X\ (\text{GoodPosition}(X) \wedge \text{NoHigherMarks}(X) \Rightarrow \text{FirstPosition}(X)) \\
\text{Rule II:} &\quad \forall X\ (\text{AttendsLectures}(X) \wedge \text{StudiesBooks}(X) \Rightarrow \text{CoversCourse}(X)) \\
\text{Rule III:} &\quad \forall X\ (\text{CoversCourse}(X) \Rightarrow \text{GoodPosition}(X))
\end{aligned}$$

Facts: $\text{AttendsLectures}(\textit{student})$, $\text{StudiesBooks}(\textit{student})$, $\text{NoHigherMarks}(\textit{student})$

7.3 Prolog Implementation (Backward Chaining)

```
% Facts
attends_lectures(student).
studies_books(student).
no_higher_marks(student).

% Rule II: Covers course if attends lectures AND studies books
covers_course(X) :-
    attends_lectures(X),
    studies_books(X).
```

```
10
11 % Rule III: Good position if covers course
12 good_position(X) :-
13     covers_course(X).
14
15 % Rule I: First position if good position AND no higher marks
16 first_position(X) :-
17     good_position(X),
18     no_higher_marks(X).
```

Listing 9: Student evaluation — backward chaining

7.4 Backward Chaining Trace

Goal: `first_position(student)`

```
first_position(student)
  |- good_position(student)  AND  no_higher_marks(student)
  |     |                              |
  |     |                              +-> FACT => TRUE
  |     |
  |     +-> covers_course(student)
  |               |
  |               +-> attends_lectures(student)  -> FACT => TRUE
  |               +-> studies_books(student)      -> FACT => TRUE
  |           => TRUE
  |
  +-> TRUE  AND  TRUE
=> TRUE   (Q.E.D.)
```

7.5 Forward Chaining Trace

Forward chaining starts from known facts and applies rules to derive new facts:

Step	Rule	Derivation
1	Rule II	`attends_lectures(student)` $\wedge$ `studies_books(student)` $\Rightarrow$ **covers_course(student)**
2	Rule III	`covers_course(student)` $\Rightarrow$ **good_position(student)**
3	Rule I	`good_position(student)` $\wedge$ `no_higher_marks(student)` $\Rightarrow$ **first_position(student)** $\blacksquare$

7.6 Forward Chaining in Prolog

```
:- dynamic derived/1.

forward_chain :-
    % Step 1: Rule II
    (   attends_lectures(S), studies_books(S),
        \+ derived(covers_course(S))
    ->  assert(derived(covers_course(S))),
        format('Step 1 (Rule II):  Derived covers_course(~w)~n'
           , [S])
    ;   true),
    % Step 2: Rule III
    (   derived(covers_course(S)),
        \+ derived(good_position(S))
    ->  assert(derived(good_position(S))),
        format('Step 2 (Rule III): Derived good_position(~w)~n'
           , [S])
    ;   true),
    % Step 3: Rule I
    (   derived(good_position(S)), no_higher_marks(S),
        \+ derived(first_position(S))
    ->  assert(derived(first_position(S))),
        format('Step 3 (Rule I):   Derived first_position(~w)~n
           ', [S])
    ;   true).
```

Listing 10: Forward chaining using assert/1

7.7 Output

```
% Backward Chaining
?- first_position(student).
true.

% Forward Chaining
?- forward_chain.
Step 1 (Rule II):  Derived covers_course(student)
Step 2 (Rule III): Derived good_position(student)
Step 3 (Rule I):   Derived first_position(student)
true.

?- derived(first_position(student)).
true.
```

Listing 11: Assignment 3 output

7.8 Discussion

This assignment explicitly demonstrates both inference strategies:

Backward Chaining Prolog starts from the goal `first_position(student)` and recursively decomposes it into sub-goals until all sub-goals are resolved to known facts. The search is top-down and goal-driven.

Forward Chaining Starting from the three known facts, rules are applied in sequence to derive new facts: first `covers_course`, then `good_position`, and finally `first_position`. The process is bottom-up and data-driven. We simulate this in Prolog using `assert/1` to dynamically add derived facts to the knowledge base.

Both strategies arrive at the same conclusion: **the student will get first position**. This confirms the soundness and completeness of the inference for this knowledge base.

8 Summary and Comparative Analysis

Table 2: Summary of all programs and inference characteristics

Program	**Key FOPL Concepts**	**Chain Depth**	**Strategy**
Program 1 (Criminal)	Universal quantification, conjunction	3 levels	Backward
Program 2 (Monkey-Banana)	Multi-predicate chaining	4 levels	Backward
Assignment 1 (Steve)	Resolution, transitive rules	2 levels	Backward
Assignment 2 (Horses)	Biconditional (inverse), inheritance	3 levels	Backward
Assignment 3 (Student Eval)	Forward + Backward comparison	3 levels	Both

8.1 Key Observations

1. **FOPL to Prolog mapping is systematic:** Universal quantifiers become Prolog variables; implications become rules (`:-`); conjunctions become comma-separated sub-goals.
2. **Backward chaining is goal-driven:** Prolog always starts from the query and works backward to facts. This is efficient when the goal space is small.
3. **Forward chaining is data-driven:** Starting from facts, new conclusions are derived iteratively. This is useful when the fact base is small and many conclusions can be drawn.
4. **Rule ordering matters in Prolog:** The sequence of clauses and sub-goals affects the search path and efficiency, even though the logical meaning remains unchanged.
5. **Inverse relations** (as in Assignment 2) are naturally expressed as bidirectional rules in Prolog.

9 Conclusion

This laboratory exercise provided comprehensive experience with First Order Predicate Logic and its computational realisation in Prolog. The key takeaways are:

1. **Knowledge Representation:** FOPL provides a rigorous formal language for representing real-world knowledge. Natural-language statements about criminals, animals, students, and course preferences were systematically translated into predicate logic and then into executable Prolog programs.
2. **Automated Reasoning:** Prolog's inference engine automatically performs resolution, unification, and backtracking to derive conclusions. The programmer only needs to specify *what* is true (facts and rules); the engine determines *how* to prove a query.
3. **Inference Strategies:** The Student Evaluation assignment demonstrated that both forward chaining (data-driven) and backward chaining (goal-driven) reach

identical conclusions, confirming the logical soundness of the knowledge base. Understanding both strategies is essential for designing expert systems.

4. **Rule Chaining:** Multi-level inference chains (as in the Monkey-Banana and Criminal problems) show how simple rules compose to solve complex reasoning problems—a foundational concept in AI knowledge-based systems.

10 References

1. Bratko, I. (2012). *Prolog Programming for Artificial Intelligence* (4th ed.). Pearson Education.
2. Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer-Verlag.
3. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
4. Nilsson, U., & Maluszynski, J. (1995). *Logic, Programming and Prolog* (2nd ed.). John Wiley & Sons.
5. SWI-Prolog Documentation. `https://www.swi-prolog.org/pldoc/`